The journal

Trunk section: §6.9 Journal.

Talks to: Sources (§6.1: the trader's Alpaca accounts, broker files, bars); the Card and Registers (§6.3, §6.6: what the desk knew about the ticker at the entry); the Market (§6.4: the market state at the entry); Quant (§6.8: the chart of a trade); Platform (§6.10: keys, schedule, backup).

Built on: legacy/journal/, documented from the code with every import format run on a scratch database: PAGE (every tab and formula), ACCOUNTS (the Alpaca accounts and their pairing), IMPORT (the file formats, what was tested), ANALYSIS (technical snapshot, conditions, grading), DATA-MODEL, OPERATIONS (start-up, the fetch timer, the tests), KNOWN-ISSUES (D-J1 to D-J20).

Contents

1. What the journal is for
2. The legacy design
3. What failed
4. The new design
5. Contracts
6. How the rebuild is proven

---

1. What the journal is for

Every real trade, after the fact: what was bought and sold, at what prices, in what size, against what stop, under what market, for what result. Then the patterns across trades: by hour, weekday, price, size, side, setup.

It answers the question the research registers do not: **how do my actual trades go?** The registers measure what screeners find; the journal measures what the trader did with it.

---

2. The legacy design

2.1 Three ways in

Way	Pairing	Id	Stop
Alpaca accounts on a schedule	running position (good)	stable: `alpaca:<account>:<first fill>`	none
File import (TradingView fills, TradingView paper log, TTP orders and trades, DXtrade)	first fill opens, first exit closes (broken)	random	none
Hand entry (API only)	—	random	optional

2.2 The Alpaca accounts (keep this design)

The trader adds an account: name (1–40 characters, unique), key id and secret (both or neither; no quotes or spaces; key ≥ 10, secret ≥ 20 characters), paper or live, fetch every 5–1440 minutes (default 30), look back 1–90 days (default 5), enabled. Keys live in a 0600 file, never in a reply, never in the backup; the page sees a shortened key id. The only calls are `GET /v2/account` (test) and `GET /v2/account/activities/FILL`. A timer each minute fetches due accounts one at a time; a failure is recorded with its reason and waits for its turn.

Fills into trades, per symbol, by running position:

Position	Becomes
0 → not 0	a trade opens; direction from this fill
further the same way	more entry; entry = **volume-weighted average**
back toward 0	exit; exit = volume-weighted average
reaches 0	the trade closes
crosses 0 in one fill	**split**: one trade closes at flat, a new one opens the other way
not 0 at the window’s end	kept as **open**, no exit

A trade is closed only when exit shares reach entry shares. **A closed trade is never rewritten** (it may carry a setup or a note).

2.3 The page and its numbers (keep the formulas)

Eight tabs (Overview, Calendar, Time, Risk, Stats, Setups, Trades, Import), three filters (account, setup, source) narrowing one list every tab draws from. Every number from closed trades with a known net P&L; winner > 0, loser < 0, a 0 trade neither but counted:

Number	Formula
win rate	winners ÷ closed × 100
profit factor	Σ winners ÷ −Σ losers (∞ with no losers)
expectancy	net ÷ closed
payoff	average win ÷ −average loss
R	net ÷ (|entry − stop| × shares)
expectancy R, SQN	mean R; mean R ÷ sample SD of R × √min(n, 100)
Kelly %	(win rate − (1 − win rate) ÷ payoff) × 100
max drawdown	largest peak − equity of cumulative net, peak from 0; % of that peak
recovery	net ÷ max drawdown
consistency	mean daily net ÷ sample SD of daily net × √252
streaks	longest runs of winners and losers in entry order

Buckets: entry hour, weekday, entry price ($0–2 … $100+), size (0–99 … 2000+ shares), side. **Insights** (5+ closed trades): buckets with 2+ trades ranked by the size of their P&L, up to two winners and two losers, with their share of all profits or losses. Plus the calendar month grid with week totals, hold times with a verdict, the R and P&L histograms, the stats catalogue, the setup table, the trade cards with a setup dropdown, a two-tap delete, and the 📈 chart link.

2.4 The parts that never worked

- **The technical snapshot** (VWAP, EMAs, Bollinger, RVOL, MAE, MFE, R, capture %, exit verdict, multi-day levels) computed only for file imports, shown nowhere.
- **Two condition systems**: 18 template rules (`alignment.js`) and a 77-check library (`checks.js`), of which 55 can never answer for a journal trade.
- **A grading engine** (setup expectancy, size multiplier, kill switch, letter grades) from the removed live trading engine, with no caller and no input.
- **17 leftover tables** from the old all-in-one app (a fresh database has 30 tables; 13 are the journal’s own).

---

3. What failed

ID	Fault (tested where marked)	Root cause
D-J1	file imports mispair scale-ins and scale-outs: 200 shares in two parts stored as 100, net $198 instead of $396 (**tested**)	first-fill pairing
D-J2	TradingView fills lose every short; tickers keep the exchange prefix (**tested**)	`sell` always an exit
D-J3	importing a file twice stores every trade twice (**tested**)	random ids
D-J4	the TradingView paper log fails with a TypeError (**tested**)	a wrong regex group index
D-J5	TTP orders history imports nothing (**tested** on the detected headers)	handed to the wrong parser
D-J6	DXtrade positions closed by hand never close (**tested**)	only market entries and stop/take-profit exits counted
D-J7	unpaired fills dropped silently	no report
D-J8	the bar window is an hour late March–November; morning trades get the day’s last bar	a fixed −05:00 offset
D-J9	no trade has a stop: R, capture, verdict and grading all empty	nothing sets one; no field
D-J10	the chart marks the trade 4–5 hours early	New York clock written as UTC
D-J11	a position older than the look-back window becomes a phantom short (**tested**)	fills before the window not fetched
D-J12	one date per trade; overnight durations wrong (**tested**)	date plus clock times
D-J13	the snapshot is mostly blank or looks ahead; 55 of 77 checks never answer	tables nothing writes; whole-day bars
D-J14	the grading half unusable; a fresh database has no setups	no writer, no reader
D-J15	no fees for Alpaca, DXtrade, the paper log; fee rules never applied	—
D-J16	14 of 29 routes uncalled; leftover schema	inherited code
D-J17	the server’s metrics differ from the page’s	two implementations
D-J18–D-J20	wrong text; missing data read as “not aligned”; small display faults	—

None of the 84 journal tests covered the file importer.

---

4. The new design

4.1 The model

Entity	Key	Holds
account	id	name, broker (`alpaca`, `file:<format>`, `manual`), paper or live, schedule, look-back, enabled, last run (keys in the secret store)
fill	account, external id	instant (UTC), symbol (canonical, no exchange prefix), side, quantity, price, fees, order id, source
trade	id (stable, §4.3)	account, symbol, direction, **entry instant**, **exit instant**, entry and exit (volume-weighted), shares, **stop**, target, gross, **fees**, net, status, setup, notes, tags
trade_fill	trade, fill, portion	which fills (or parts of a fill) make the trade
setup	id	name, description, enabled (editable on the page)
trade_snapshot	trade	the desk's knowledge at the entry (§4.5)

4.2 One pairing engine

Every source produces **fills**; one engine pairs them into trades with the running-position rules of §2.2, per account and symbol, in instant order. A partial fill is a fill. Scale-ins, scale-outs, reversals and open positions are handled once, for every source.

Positions older than the look-back window (D-J11): each fetch starts from the account's stored open positions (or fetches back to the oldest open position's first fill), so a closing fill always finds its opening.

4.3 Stable ids

Source	Id
Alpaca	`alpaca:<account>:<first fill id>` (kept)
a file	`file:<account>:<symbol>:<first fill instant>:<first fill price>:<qty>`, or the broker's order id when the file has one
hand entry	a random id, once

Importing the same file twice **updates**; it never duplicates (D-J3). A closed trade the trader has touched (setup, note, stop) is never overwritten by a later fetch; a change in its fills is shown as a conflict for the trader to accept.

4.4 File formats: one parser each, each with a sample and tests

Format	Detected by	Parser rules
TradingView fills	`Symbol`, `Side`, `Qty`, `Price`, `Date`	strip the exchange prefix; `Buy` / `Sell` are fills, and the pairing engine decides what opens and closes (so shorts work)
TradingView paper log	`Time`, `Text`	executed lines are fills; the side comes from the matching “call to place” line, correctly indexed
TTP orders history	`Date/Time`, `Order ID`, `Event`	its own parser, reading its own columns
TTP trades (older)	`Action`, `Shares`	as legacy
DXtrade orders	`;` with `Symbol`, `Side`, `Event`	every `Filled` order of any type is a fill; side from the order side

Each import answers with what it used **and what it did not**: fills read, trades made, open positions, rows skipped with reasons (D-J7). A sample file per format, with shorts, scale-ins, scale-outs and partial closes, runs in CI.

4.5 The stop, the fees, the snapshot

- **Stop**: from the broker’s stop order when there is one (Alpaca orders API); from the strategy’s exit plan when the trade came from a quant signal; else entered on the trade card. A trade without a stop shows “no stop · R unknown” and is counted apart (D-J9).
- **Fees**: from the broker where reported; else from the account’s fee rule, applied and labelled as estimated (D-J15). Editable.
- **Snapshot** at the entry instant, from nothing later than the entry:
  - the **price structure** (VWAP, EMAs, levels, the day’s range so far, RVOL, ATR) computed by the card builder’s own functions from 1-minute bars up to the entry minute and daily bars before the day;
  - **short interest, borrow, news, catalyst and the CANSLIM reading** from the fact store as known at the entry (`known_at` ≤ entry). A ticker no tool carded that day has none, so the journal asks the adapters for them with their own dates (a settlement date, a quarter, a headline’s time) and keeps only what is dated before the entry; the rest is `unavailable`, said so;
  - the **market state** current at the entry, and the **R1 row** when the ticker was on a tool that day.

MAE and MFE come from 1-minute bars between the entry and exit instants (on the right dates, from the bar adapters). No look-ahead, no table nothing writes (D-J8, D-J13).

4.6 Analysis

- The page's formulas (§2.3) are kept and computed **in one place** (the server, with the page rendering), so a number cannot differ between two implementations (D-J17).
- **One condition system**: rules over the snapshot's fields (the card's field names), written with the screener builder's operators (TOOLS-AND-SCREENERS.md §4.1); only checks whose fields the snapshot holds are offered. It replaces both legacy systems (the 18 template rules and the 77-check library).
- **Setup statistics** (expectancy in $ and R, win rate, payoff, drawdown, streaks) per setup on the Setups tab, from trades with stops. The legacy grading engine (size multipliers, kill switch) is **not carried over**: the desk places no orders.

4.7 The page

The legacy page's layout, tabs, filters and formulas are kept. Added: a stop, target, fees, notes and tags on each trade card; a setup editor; the snapshot and MAE/MFE on the trade card; day notes on the calendar; the import report. **The chart button** opens quant at the trade's **real instants** (D-J10). Every label has its ⓘ.

---

5. Contracts

Owns

Table	Key
`accounts`	id
`fills`	account, external id
`trades`	id
`trade_fills`	trade, fill
`setups`	id
`trade_snapshots`	trade
`day_notes`	day

Reads

From	What
sources	account fills and orders (Alpaca), bars, broker files
card, registers, market	facts, R1 rows, market states as of the entry
platform	keys, the schedule

Routes (desk-web)

Route	Does
`GET /api/journal/trades` (filters)	trades
`GET /api/journal/trades/:id`	a trade with its fills and snapshot
`PATCH /api/journal/trades/:id`	stop, target, fees, setup, notes, tags
`POST /api/journal/trades`	hand entry
`POST /api/journal/import`	a file → import report
`GET/POST/DELETE /api/journal/accounts…`, `…/test`, `…/fetch`	accounts (no secret ever returned)
`GET/POST/PATCH /api/journal/setups`	setups
`GET /api/journal/stats` (filters)	every number of §2.3, server-computed

---

6. How the rebuild is proven

1. **Import tests.** Every case that failed in the legacy test (scale-in and out, a TradingView short, the paper log, TTP orders, a DXtrade hand close, a re-import) gives the right trades.
2. **Re-import of history.** Every legacy Alpaca trade is re-derived from its fills by the new engine and compared; differences are explained (old phantom shorts, overnight durations).
3. **Instant tests.** A June morning trade’s snapshot and chart marks land on the right bar.
4. **Formula parity.** The page’s legacy formulas, run on the same trades, give the same numbers as the server’s.